

BRiLaunch

Plateforme dynamique de services

1. Introduction :

BRiLaunch est une plateforme Java qui permet de gérer des services dynamiques. Elle est accessible à deux types d'utilisateurs :

- **Les clients amateurs** : Ils utilisent les services proposés par la plateforme.
 - **Les clients programmeurs** : Ils ajoutent leurs propres services et mettent à jour leurs services.
-

2. Ce qui a été réalisé :

1. Lancement du serveur : appli.BRiLaunch.

La classe **BRiLaunch** initialise l'application côté serveur. Elle lance dynamiquement des serveurs, chacun configuré avec un port et un service spécifiés explicitement dans la variable args.

Les classes des services sont chargées dynamiquement et enregistrées dans le outilServer.ServiceManager avec leur numéro de port.

Par la suite, les serveurs sont créés et configurés pour écouter sur ces ports spécifiques.

Usage de l'application : `java BRiLaunch <ClasseService1> <Port1> [<ClasseService2> <Port2> ...]`

2. Découplage entre le serveur et ses services grâce au package outilServer

outilServer.Serveur :

Classe serveur générique permettant de créer un serveur écoutant sur un port, cette classe sollicite le ServiceManager pour obtenir le service associé à son port.

outilServer.ServiceServeur :

Classe abstraite représentant un service générique côté serveur. Elle fournit un modèle de base pour les services, qui doivent être sous-classés en redéfinissant la méthode run() afin d'être pleinement opérationnels.

outilServer.ServiceManager :

Classe responsable de gérer l'association entre les services et les ports. Elle maintient une liste des classes de service, qui doivent obligatoirement implémenter la classe abstraite outilServer.ServiceServeur pour être enregistrées.

À tout moment, un serveur peut demander le service qui lui est assigné en fournissant son port, permettant ainsi une gestion dynamique et centralisée des services.

3. Classe Programmeur :

Logique pour préserver les pseudos, mot de passe et url vers un serveur ftp pour les programmeurs.

Cette classe contient également une liste de numéros de service, permettant à chaque programmeur de retrouver et de gérer ses propres services en interrogeant ServiceRegistry.

4. Ressource partagée entre programmeurs et amateurs : ServiceRegistry.

Une base de données en mémoire qui :

- Stocke les services enregistrés par les programmeurs.
- Respect la norme BRi
- Facilite l'accès rapide aux services pour les amateurs.

5. La Norme BRi :

La Norme BRi est comme son nom, une norme permettant de **vérifier la conformité** des classes de service BRi entrantes, comprenant :

- Implémenter l'interface BRi.Service
- ne pas être abstract

- être publique
- avoir un constructeur public (socket) sans exception
- avoir un attribut Socket private final
- avoir une méthode public static string toStringue() sans exception

La structure du code emploie :

- java.lang.reflect : permettant d'analyser dynamiquement les classes, champs, constructeurs et méthodes

Après toute ces vérifications, la classe est ajoutée à la liste des services :

services.add(classService)

6. Application Cliente Amateur (clientama) :

Application qui permet à un utilisateur amateur de se connecter à la plateforme BRiLaunch via le port 3000 et d'entrer en connexion avec le **ServiceBRiAma**. **Ce service permet** aux amateurs de consulter la liste des services et utiliser ces services

7. Application Cliente programmeur (clientprog) :

Application qui permet à un utilisateur amateur de se connecter à la plateforme BRiLaunch via le port 3000 et d'entrer en connexion avec le **ServiceBriProg**.

8. ServiceBRiAma:

Service côté serveur qui gère les amateurs et leurs interactions avec les services des programmeurs disponibles sur la plateforme.

9. ServiceBriProg:

Service côté serveur dédié à la gestion des programmeurs, leur permettant d'ajouter, mettre à jour et gérer leurs services via leur serveur FTP.

3. Ce qui marche :

Pour les amateurs via **ServiceBRiAma** :

- Consultation de la liste des services disponibles, lancement d'un service.

Une liste de numéros de service associés à leurs noms est affichée grâce à ServiceRegistry.

L'utilisateur amateur est invité à choisir un service en entrant son numéro.

Le service sélectionné est récupéré depuis le ServiceRegistry et exécuté.

- **Services prise en charge :**
 - Service simple : Inversion d'un texte ou tout autre service manipulant des textes String via la socket.

Pour les programmeurs via **ServiceBriProg** :

- **Authentification/Inscription du programmeur** : Un programmeur s'inscrit avec un pseudo, mot de passe et url de son serveur ftp. il se connecte avec son pseudo et mot de passe

Le programmeur est invité à taper **"I"** pour s'inscrire ou **"C"** pour se connecter.

- **Gestion des services avec serveur Ftp :**

Une fois connecté, un menu propose les options suivantes :

1. **Fournir un nouveau service**
2. **Mettre à jour un service**
3. **Changer l'adresse FTP**
4. **Quitter**

En saisissant un numéro (1, 2, 3 ou 4), le programmeur peut accéder directement à l'onglet correspondant.

1. Fournir un nouveau service :

- Le programmeur est invité à indiquer le nom du nouveau service disponible sur son serveur FTP.
- Il doit spécifier le nom complet de la classe en respectant la convention `login.nomDuService` (par exemple : `matthieu.ServiceInversion` pour un utilisateur ayant le login `matthieu`).
- La classe est chargée depuis son serveur FTP, puis ajoutée au ServiceRegistry après une vérification de la norme BRi.

2. Mettre à jour un service :

- Une liste de ses services, avec leurs numéros, est affichée.
- Le programmeur sélectionne un service en entrant son numéro.

- Il fournit ensuite le nom de la nouvelle classe correspondant à la mise à jour.
- La mise à jour est appliquée au ServiceRegistry, et les amateurs peuvent en bénéficier sans redémarrer l'application.

3. Changer l'adresse FTP :

- Le programmeur saisit la nouvelle URL de son serveur FTP.

Les fonctionnalités répondent au minimum au cahier des charges.

Les programmeurs peuvent ajouter ou mettre à jour des services sans avoir à redémarrer BRiLaunch, ce qui permet une gestion dynamique et continue des fonctionnalités.

4. Amélioration possible

Extension de la fonctionnalité des services :

- Ajouter de nouveaux services dans le système, comme des services avec ressource partagée et d'analyse de fichier xml.

Extension des applications clientes:

- Création d'une classe de référence pour les clients, qui centralise certaines fonctionnalités communes aux deux applications clientes, telles que la gestion de la connexion au serveur.